# PQC Project – Phase 1 & 2 Explained Simply

Oct 8, 2026 · @Adarsh

## 1. The problem, in simple words

Today's internet security depends on algorithms called **RSA** and **ECC**. They protect banking, WhatsApp, logins and cloud data. They are safe because normal computers would take thousands of years to break them.

A powerful **quantum computer** will break them in hours, using a method called **Shor's algorithm**. The day this becomes possible is called **Q-Day**. Nobody knows the exact year, but most experts expect it within the next 10 to 20 years.

The scary part is **"Harvest Now, Decrypt Later"**. Attackers can record encrypted data *today*, store it, and simply decrypt it after Q-Day. So data that must stay secret for many years, like medical records or government IDs, is already at risk right now.

In August 2024, NIST (the US body that sets security standards) released new quantum-safe algorithms: **ML-KEM** (for exchanging keys) and **ML-DSA** (for digital signatures). These are called **Post-Quantum Cryptography (PQC)**.

**Important:** PQC does not use quantum computers. It is normal software that runs on normal computers, designed so that even a quantum computer cannot break it. Our domain is **cybersecurity / cryptography**, not quantum computing.

## 2. What our project is, and what is our own contribution

The new algorithms already exist, so we do **not** invent new algorithms. Writing your own cryptography is a known security mistake. We use the official NIST algorithms through **liboqs**, the standard open-source library used by researchers and companies.

The real problem companies face is not "which algorithm", but **how to move to PQC safely and cheaply**. Our framework solves four practical problems:

1. **What if the new algorithm has a hidden flaw?** We use a **hybrid**: old algorithm (X25519) + new algorithm (ML-KEM) together. An attacker must break both.
2. **What if an algorithm gets broken later?** Our system is **crypto-agile**: the algorithm is chosen from a settings file, and if one is declared broken, the system automatically switches to a backup without changing code.
3. **How do we move old stored data to PQC?** Instead of re-encrypting terabytes of data, we only **re-wrap the small key** that protects it. Fast and cheap.

4. **Which data actually needs PQC?** Not all data needs heavy protection. An OTP lives 5 minutes; a medical record must stay secret 50 years. Our **Mosca risk policy engine** decides, for each type of data, how much protection it needs and which data to migrate first.

**Our main contribution is point 4, the Mosca risk policy engine.** It is based on Mosca's theorem: if (how long data must stay secret) + (time to migrate) > (years until Q-Day), the data is already at risk today. We turned this idea into a working engine and measured that it protects all at-risk data while using about 80% less extra data than putting PQC on everything.

Simple way to say it: *"The algorithms are the engine. We built the system that decides where to use it, how to switch it safely, and how to upgrade old data cheaply."*

## 3. Phase 1: the crypto core (done)

Phase 1 built the foundation: all the algorithms, the hybrid, and the switching system, plus measurements.

| What we built | In simple words | File |
|---|---|---|
| Classical algorithms (X25519, RSA-3072, Ed25519) | Today's algorithms, kept for comparison and for the hybrid | `pqcf/kem.py`, `pqcf/signature.py` |
| Post-quantum algorithms (ML-KEM-512/768/1024, ML-DSA-44/65) | The new NIST quantum-safe algorithms, used through liboqs | `pqcf/kem.py`, `pqcf/signature.py` |
| Hybrid key exchange (X25519 + ML-KEM-768) | Both algorithms run together; their two secrets are mixed into one key using HKDF. Attacker must break both | `pqcf/kem.py` |
| AES-256-GCM encryption | Encrypts the actual data. AES-256 is already quantum-safe | `pqcf/aead.py` |
| Crypto-agility registry | All algorithms are registered by name. The active one is chosen in `config.json`, so switching needs no code change | `pqcf/registry.py` |
| Benchmark | Measures speed and size of every algorithm | `benchmark.py` |

| What we built | In simple words | File |
|---|---|---|
| Demo | Client and server create a shared secret, encrypt a message, detect tampering | `demo_phase1.py` |

A common interface is the key idea: every algorithm, old or new, has the same three functions (create keys, lock a secret, unlock a secret). That is why we can swap algorithms freely.

## 4. Phase 2: the smart part (done)

Phase 2 built the four features that make this more than "just using an algorithm".

**a) Mosca risk policy engine** ( `pqcf/policy.py` ) — our main contribution

For each type of data, it calculates: shelf life + migration time − years to Q-Day. If the result is above zero, the data is already at risk. Then it picks the protection:

| Data | Must stay secret for | Decision |
|---|---|---|
| OTP code | 5 minutes | Classical is enough |
| Login session | 1 day | Classical is enough |
| Chat message | 2 years | Classical is enough |
| Bank transactions | 10 years | Hybrid (ML-KEM-768) |
| Legal contract | 25 years | Hybrid-High (ML-KEM-1024) |
| Medical record | 50 years | Hybrid-High (ML-KEM-1024) |
| Government ID | 70 years | Hybrid-High (ML-KEM-1024) |

(Assumption used: Q-Day in 12 years, migration takes 3 years. Both can be changed.)

It also creates a **migration order**: most at-risk data first (Government ID, then Medical, then Legal, then Bank).

**b) Key re-wrapping for stored data** ( `pqcf/envelope.py` )

Each stored file is encrypted with its own small key (32 bytes). That small key is then "locked" with the public-key algorithm. To upgrade a file to PQC, we only re-lock the small key. The big encrypted data is never touched. Think of it as changing the lock on a safe instead of moving everything into a new safe.

**c) Algorithm-broken simulation** ( `pqcf/registry.py` )

We can declare "ML-KEM is broken". The system immediately blocks ML-KEM and every hybrid using it, and automatically switches to the backup **FrodoKEM**, a quantum-safe algorithm based on different mathematics. No code change, no downtime. Old data can still be read so it can be moved.

**d) Protection against downgrade attacks** (`pqcf/negotiation.py`)

When client and server agree on an algorithm, an attacker in the middle can delete the strong options so both sides fall back to the weak one. Our server digitally signs (with ML-DSA-65) exactly what the client offered. If anything was changed on the way, the client detects it and stops the connection.

**Also:** `demo_phase2.py` shows all four features, `evaluate_phase2.py` produces the results below, and **31 automatic tests** check everything works.

## 5. Our results in plain numbers

The headline result: **our risk-aware policy protects every at-risk record while using about 80% less extra data than putting PQC on everything.** All numbers below come from our own test runs (saved in the `sample_output` folder).

**Finding 1: PQC is fast, but big.**

| Algorithm | Quantum-safe | Time per operation | Data sent |
|---|---|---|---|
| X25519 (today) | No | 0.08 ms | 64 bytes |
| RSA-3072 (today) | No | 1.7 ms (unlock) | 806 bytes |
| ML-KEM-768 (new) | Yes | 0.02 ms | 2,272 bytes |
| Hybrid X25519 + ML-KEM-768 | Yes | 0.13 ms | 2,352 bytes |
| FrodoKEM-976 (backup) | Yes | 0.85 ms | 31,376 bytes |

Surprise: the new ML-KEM is even **faster** than today's algorithms. The real cost is **size**: about 37 times more data than X25519. That is exactly why we should not use PQC everywhere blindly.

**Finding 2: the risk policy saves a lot (10,000 sample data records).**

| Strategy | Extra key data | At-risk records left unprotected |
|---|---|---|
| Classical everywhere | 0.64 MB | 1,473 (unsafe) |
| PQC everywhere (ML-KEM-768) | 23.52 MB | 0 |

| Strategy | Extra key data | At-risk records left unprotected |
|---|---|---|
| PQC everywhere (ML-KEM-1024) | 32.16 MB | 0 |
| **Our Mosca policy** | **4.62 MB** | **0** |

The sample mix (30% OTPs, 25% sessions, 20% chats, down to 1% government IDs) is our assumption. With more long-lived data, the savings would be smaller.

**Finding 3: re-wrapping is far cheaper than re-encrypting.** Re-wrapping always takes about 0.22 ms, whatever the file size. Re-encrypting grows with size: 142 ms for a 50 MB file, about 630 times slower.

**Finding 4: the downgrade attack is real.** A normal client was silently pushed down to quantum-breakable X25519 by our simulated attacker. Our protected client detected the attack and stopped.

## 6. What is still left (Phase 3)

13 of 16 planned components are done. The remaining 3 are about showing the system working like a real cloud service:

- [ ] **Client and server services** (FastAPI) that talk to each other using our framework
- [ ] **Web dashboard** with charts and a "Harvest Now, Decrypt Later" visualiser, where you enter a data type and see the policy decision live
- [ ] **Docker packaging**, a final evaluation on a more realistic workload, and the project report

## 7. How to present to ma'am (about 10 minutes)

1. **The problem (1 min).** Explain Section 1 in your own words: quantum computers will break RSA, and attackers are already recording data to decrypt later.
2. **Our idea (1 min).** "We don't invent algorithms. We built a framework that decides which data needs quantum protection, switches algorithms safely, and upgrades old data cheaply."
3. **Show the code structure (1 min).** Open the project in VS Code. Show the `pqcf` folder: one file per feature (see the tables in Sections 3 and 4).
4. **Phase 1 (2 min).** Open `sample_output/demo_phase1_output.txt`. Point at: the list of algorithms, "Secrets match: True", the algorithm switch, and "tampered ciphertext rejected".

5. **Phase 2 (3 min).** Open `sample_output/demo_phase2_output.txt`. Walk through its 4 parts: the Mosca table, re-wrapping ("payload unchanged: True"), ML-KEM broken → switched to FrodoKEM, and the downgrade attack detected.
6. **Results (1 min).** Show the two tables from Section 5. The one line to say: "Our policy protects all at-risk data with 80% less extra data than PQC everywhere."
7. **Tests and next steps (1 min).** Open `sample_output/tests_output.txt` (31 passed), then read the Phase 3 list.

**If ma'am asks to run it live:** the library (liboqs) needs a Linux environment to compile, which is set up on Adarsh's laptop. Say the live run will be shown in the next review; the saved outputs are from a real run of this same code. Do not claim it is running if it is not.

## 8. Questions ma'am may ask, with simple answers

**"The algorithms already exist. What did you do?"** We didn't make algorithms, we made the system around them: deciding which data needs quantum protection (Mosca policy), switching safely when an algorithm breaks, upgrading stored data cheaply, and blocking downgrade attacks. Like a car: the engine exists, we built the car that uses it safely.

**"Is this quantum computing?"** No. It's cryptography that runs on normal computers but resists quantum attacks. The word "quantum" refers to the threat, not the technology we use.

**"Why not just use ML-KEM everywhere?"** Two reasons. It's new (2024), so we combine it with X25519 in case it has a hidden flaw. And it sends about 37 times more data, so using it where it's not needed wastes bandwidth. Our policy uses it where it matters.

**"What is Mosca's theorem?"** If the years data must stay secret plus the years needed to migrate is more than the years until Q-Day, that data is already at risk. Example: medical records (50 years) + 3 years migration > 12 years to Q-Day, so they are at risk today.

**"How do you know when Q-Day is?"** Nobody knows exactly. That's why it's a setting in our engine. We tested three scenarios: 8, 12 and 20 years.

**"Is AES also broken by quantum computers?"** No. Quantum computers only weaken AES-256 to the strength of AES-128, which is still safe. It's RSA and ECC that get fully broken. That's why we only re-wrap keys, never re-encrypt the data.

**"Does re-wrapping protect data already stolen?"** No, and we say that honestly. Re-wrapping protects data we still store. Data an attacker already captured while it was protected by old algorithms cannot be saved. That's why migrating early matters.

**"Why FrodoKEM as the backup?"** It's quantum-safe but built on different mathematics from ML-KEM, so a weakness in ML-KEM is unlikely to affect it. It's much bigger (about 31 KB), so it's only a fallback.

**"What is a downgrade attack?"** An attacker in the middle deletes the strong options from the client's list, so both sides end up using a weak algorithm without knowing. Our server signs what it received, so the client detects any change.

**"Is the 80% saving real?"** It's measured on a sample of 10,000 records with an assumed mix of data types. With more long-lived data the saving is smaller. In Phase 3 we will test a more realistic workload.

## 9. Simple glossary

| Term | Meaning in one line |
|---|---|
| RSA / ECC / X25519 | Today's public-key algorithms; broken by quantum computers |
| Shor's algorithm | The quantum method that breaks RSA and ECC |
| Q-Day | The day a quantum computer can break today's encryption |
| Harvest Now, Decrypt Later | Recording encrypted data today to decrypt it after Q-Day |
| PQC | Post-Quantum Cryptography: normal software that resists quantum attacks |
| ML-KEM (Kyber) | NIST 2024 standard for securely sharing a secret key |
| ML-DSA (Dilithium) | NIST 2024 standard for digital signatures |
| FrodoKEM | Backup quantum-safe algorithm based on different maths; bigger |
| KEM | Key Encapsulation: a way to safely send a secret key to someone |
| Hybrid | Old + new algorithm together, so an attacker must break both |
| HKDF | A method to mix two secrets into one strong key |
| AES-256-GCM | Fast encryption for the actual data; stays safe against quantum |
| Crypto-agility | Ability to switch algorithms without rewriting the system |

| Term | Meaning in one line |
| --- | --- |
| Key re-wrapping | Upgrading only the small key that protects data, not the data itself |
| Mosca's theorem | Data is at risk if (secret lifetime + migration time) > years to Q-Day |
| Downgrade attack | Tricking both sides into using a weaker algorithm |
| liboqs | The standard open-source library of NIST post-quantum algorithms |